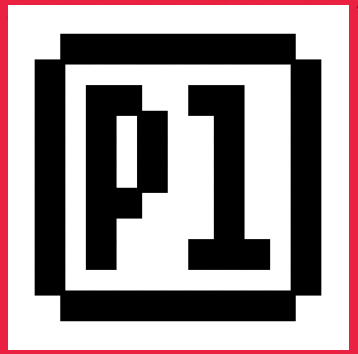
Programación en C



Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

r33

01 / 2026

Pendientes 2025

- conversiones explícitas (casts)
- conversiones implícitas (por asignación)
-



¿Por qué C?



¿Qué barco es? ¿Qué función cumple?

{ Existen múltiples lenguajes que *técnicamente* son mejores }

Pero vamos a programar sin las rueditas de entrenamiento



Un poco de contexto



1972

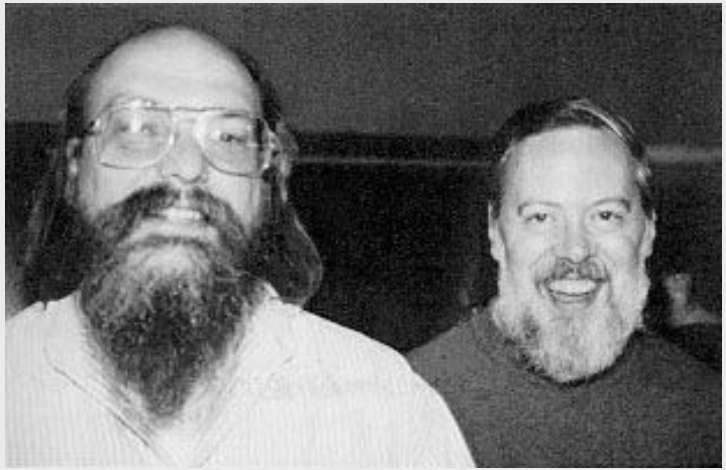
Creado por

Dennis Ritchie

y

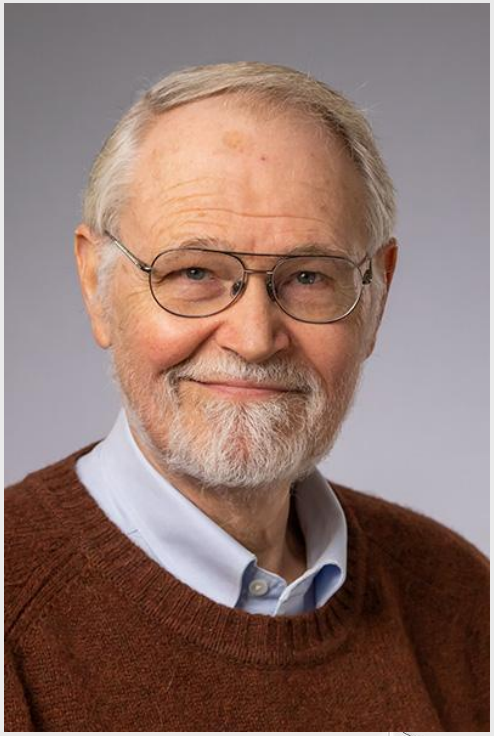
Brian Kerningham

¡~50 años!



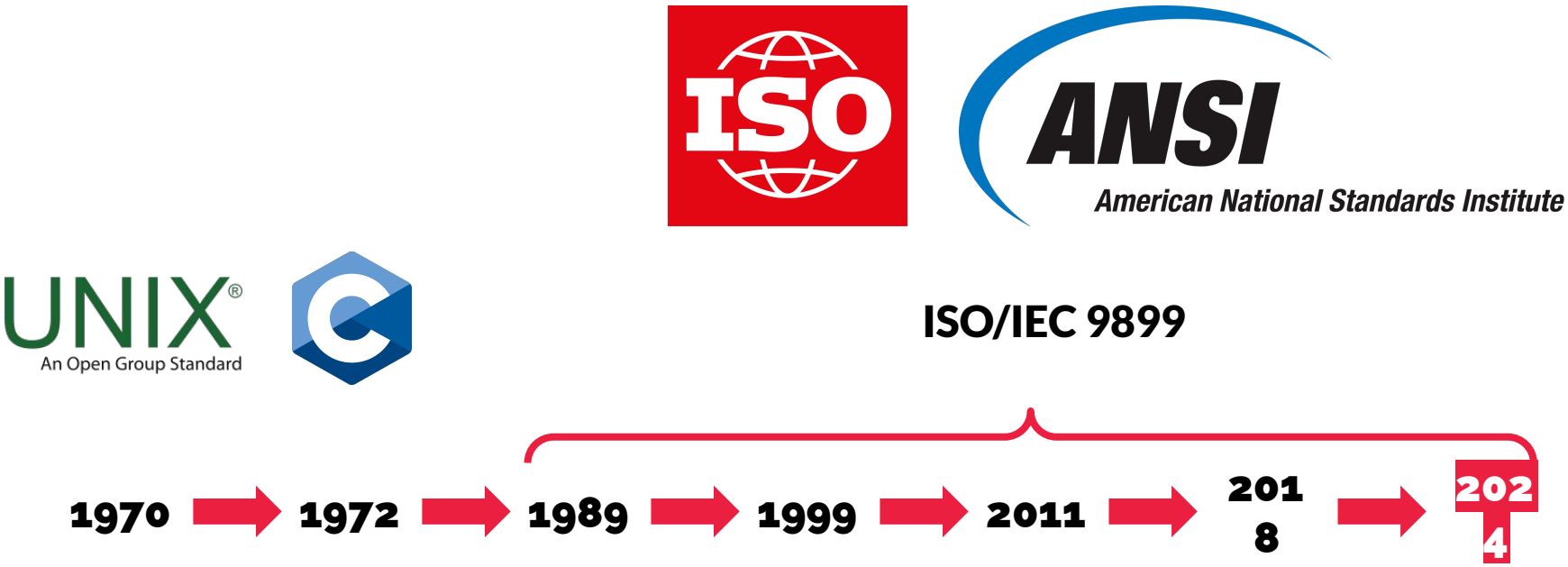


Dennis Ritchie
1941-2011



Brian Kerningham

– En revisión constante



https://www.iso.org/standard/82075.html

Un lenguaje muy influyente

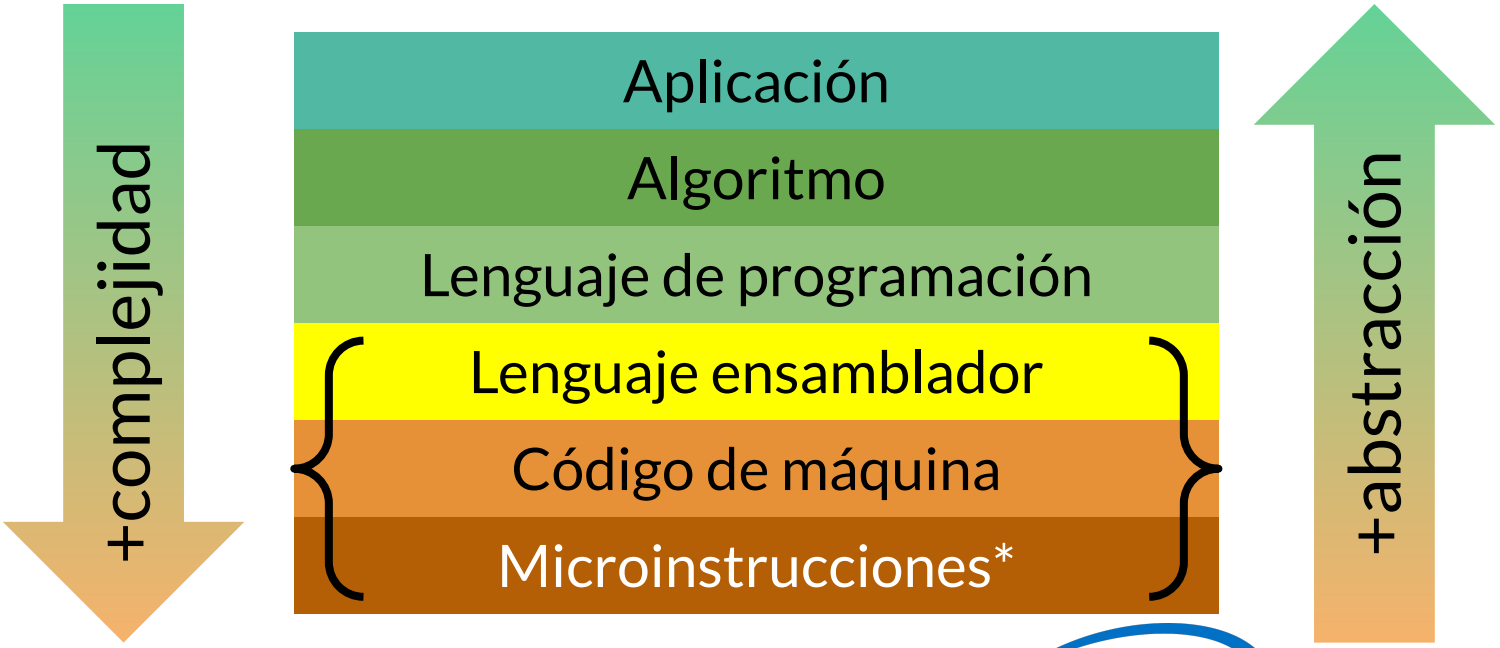
- C++
- C#
- Javascript
- PHP
- Objective-C
- Java

era

de alto nivel

cuando salió

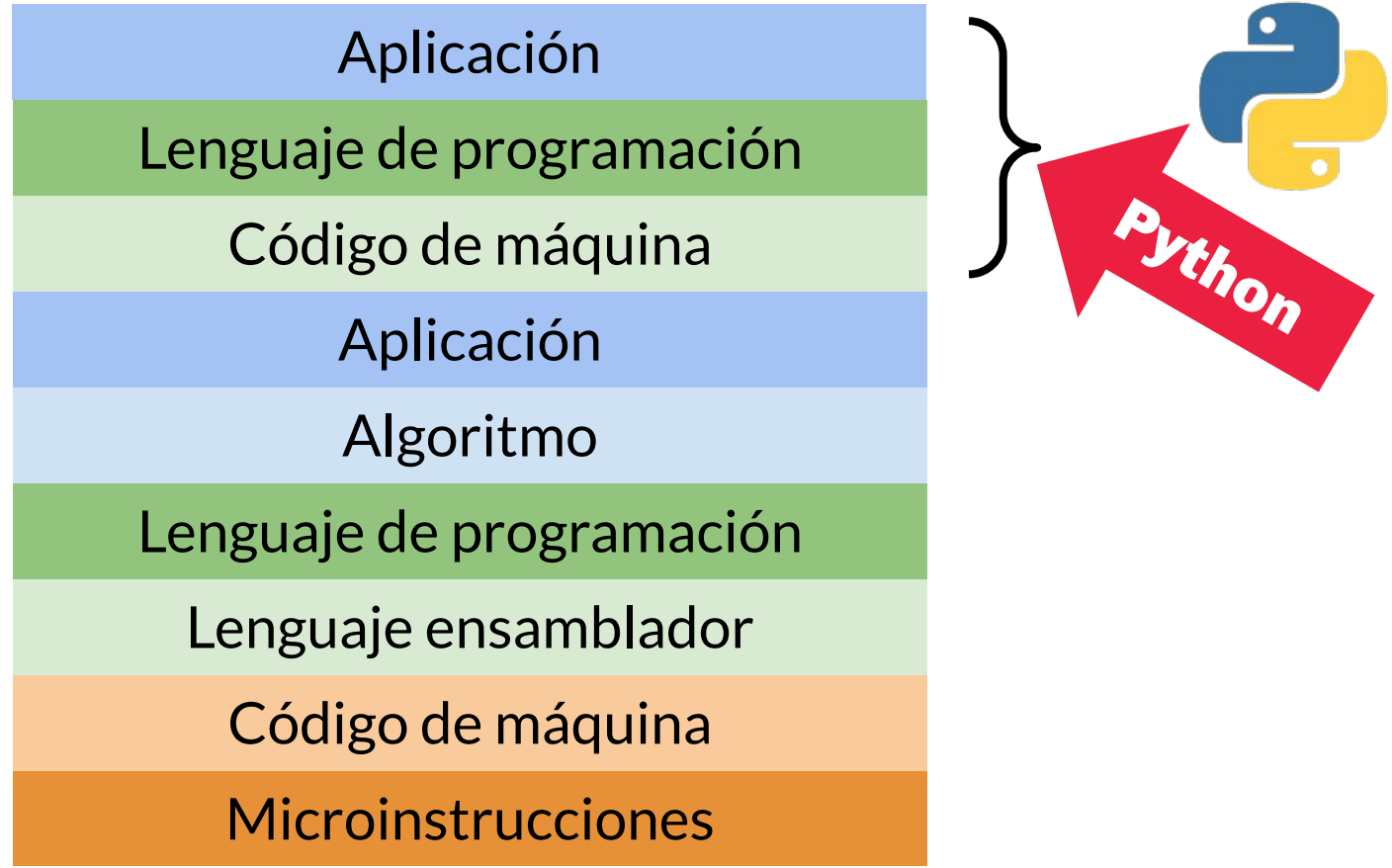
Niveles de abstracción [tradicionales]



es considerado

de nivel medio

Niveles de abstracción



Esta es la distinción entre compilado e interpretado

De propósito general*

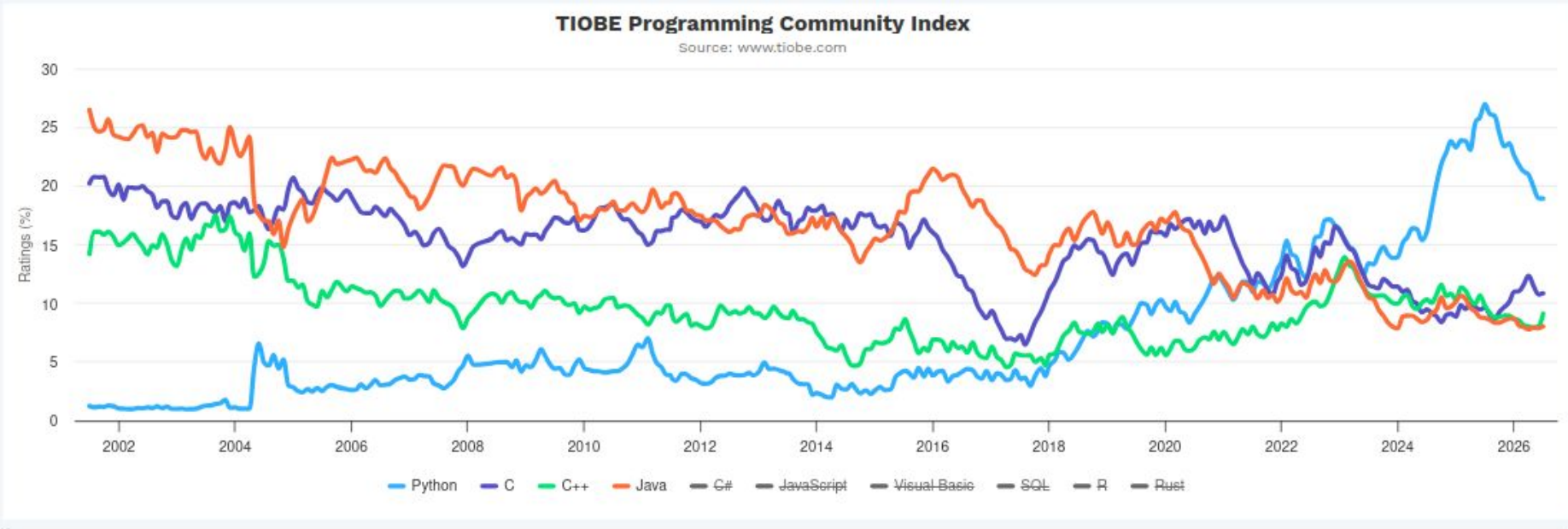
para cuándo salió

simple

Altamente eficiente

Altamente portable

1 - Python
2 - C
3 - C++
4 - Java



¡Bastante popular! (#2)

UNRN Universidad Nacional de Río Negro

https://www.tiobe.com/tiobe-index/


¿Preguntas?

Imperativo

Las instrucciones cambian el estado del programa

Imperativo* es:

Que las instrucciones sean **secuenciales** con **control de flujo** que lo manipula para modificar el **estado** del programa.

Estructurado

Las instrucciones se agrupan en unidades independientes* entre sí.

Estructurado en

Funciones y procedimientos, que se diferencian entre sí solo por la forma de interactuar con el estado del programa.


¿Preguntas?

Tenemos que ver

sintaxis

La estructura válida de un programa
compila o no compila

gramática

Un programa correctamente escrito que
no funciona como se espera

estilo

Un programa que funciona correctamente, pero estableciendo reglas sobre la sintaxis y gramática


¿Preguntas?

Empecemos por el hola mundo

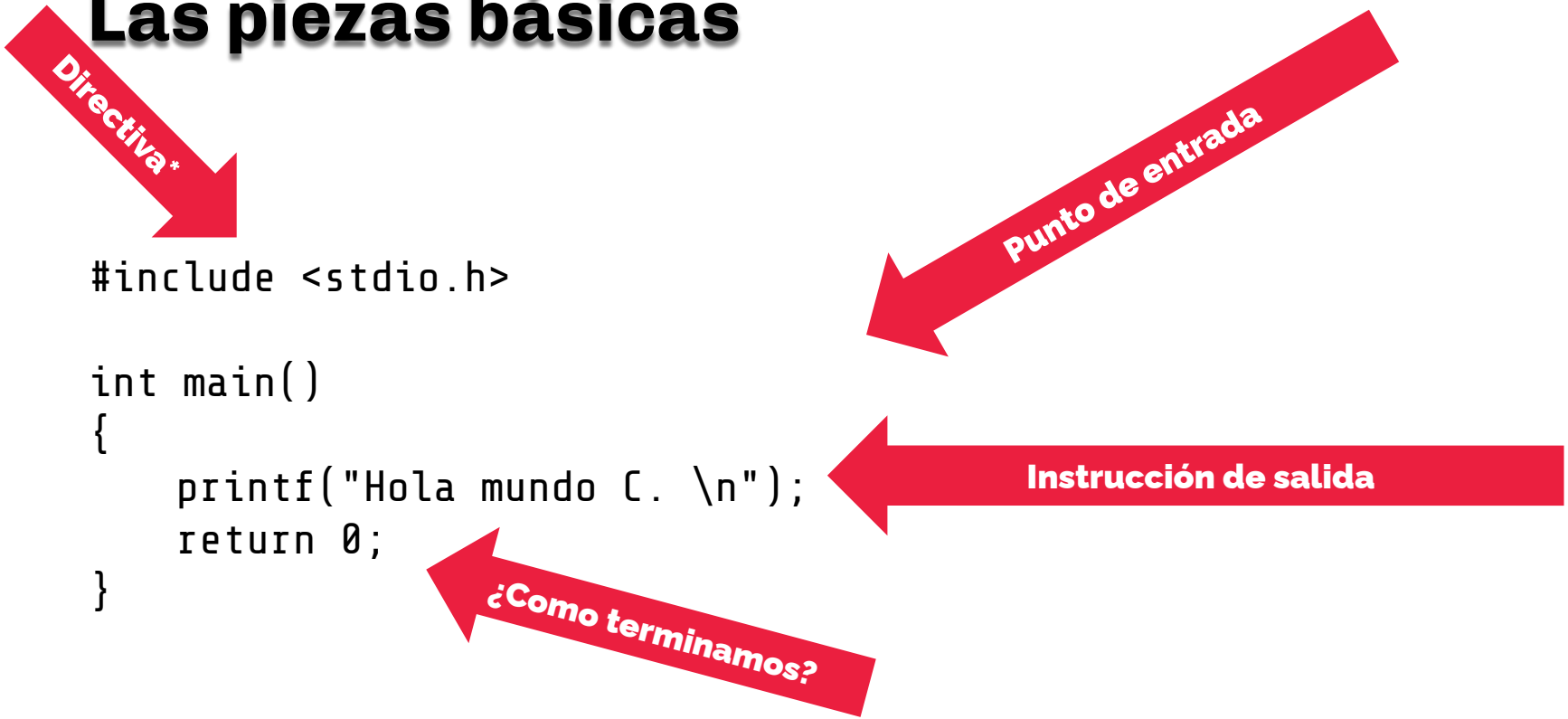
UNRN Universidad Nacional de Río Negro

Un “Hola Mundo C.”

```c
#include <stdio.h>

int main()
{
    printf("Hola mundo C. \n");
    return 0;
}
```

Las piezas básicas




```c
#include <stdio.h>

int main()
{
    printf("Hola mundo C. \n");
    return 0;
}
```

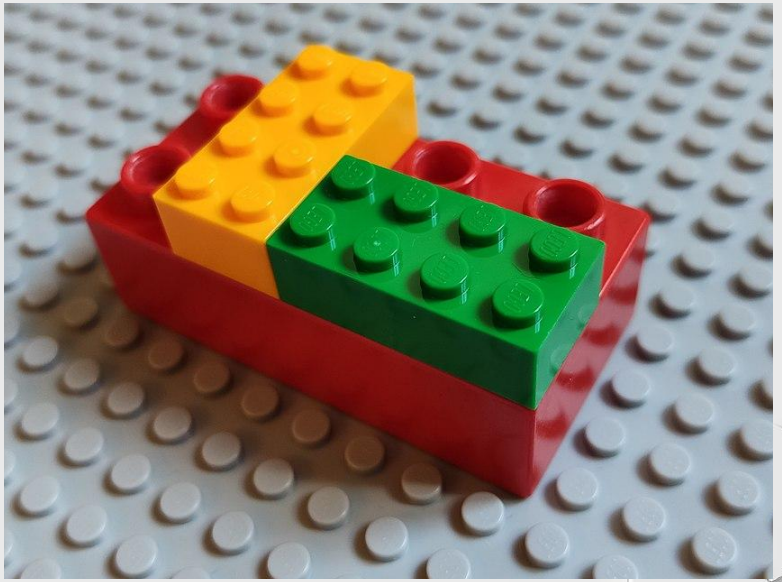
Compilando y ejecutando

```
$> gcc hola.c
$> ./a.out   (linux)
$> ./a.exe   (windows)
```

¡A la terminal!



Empezando por los bloques más simples



Paso a paso

UNRN Universidad Nacional de Río Negro

1 - El punto de entrada

```
#include <stdio.h>

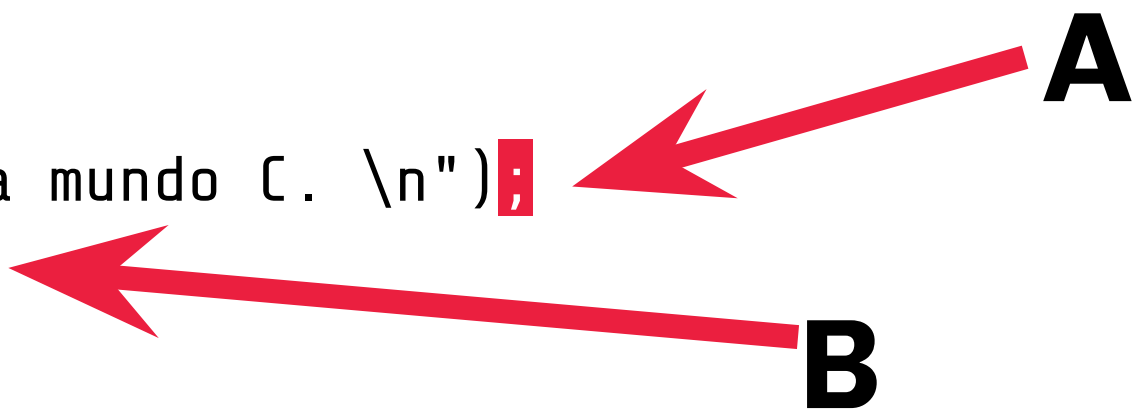
int main()
{
    printf("Hola mundo C. \n");
    return 0;
}
```



2 - Sentencias

```c
#include <stdio.h>

int main()
{
    printf("Hola mundo C. \n");
    return 0;
}
```



3 - Bloques



```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hola mundo C. \n");
    return 0;
}
```

¡A la terminal!



4 - ¡esto compila!

¡Cuestiones de estilo!

```
#include <stdio.h>
int main(){printf("Hola mundo C. \n");return 0;}
```

La claridad y prolijidad son muy importantes


¿Preguntas?

Sintaxis

UNRN Universidad Nacional de **Río Negro**

Identificadores

o coloquialmente, nombres

Deben empezar con `azAZ_` y pueden contener luego `09`.

Identificadores correctos y no válidos

valido	incorrecto
`suma_cuadrados`	
`sumaCuadrados`	`2SumaCuadrados`
`SumaCuadrados`	`2Suma-2`
`SUMA_CUADRADOS`	`int`
`SUMA2`	



Palabras reservadas

```
auto        register    sizeof
double      typedef     volatile
int         char        do          static while
struct      extern      if          _Bool
break       return      continue    _Imaginary
else        union       for         restrict
long        const       signed      _Complex
switch      float       void        inline
case        short       default
enum        unsigned    goto
```

Palabras reservadas en el Hola Mundo C.

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hola mundo C. \n");
    return 0;
}
```

Los identificadores deben ser descriptivos**

Variables

UNRN Universidad Nacional de Río Negro

A diferencia de Python

```
int identificador;
int identificador = 0;
```

Las variables van con tipo.

Variaciones *

```
int a;
```

sin inicializar

```
int b = 10;
```

inicializada en ‘10’

```
int c, d, e;
```

tres variables sin inicializar

```
int f = 1, g = 4;
```

dos variables inicializadas

`int` es para número entero (ahorita pasamos a ese tema)

Una declaración de variable por línea

Siempre inicializar la variable a un valor conocido

¿Qué tipo de datos hay?

Números enteros

opcionalmente `unsigned`

```
short int          [16-bit]
int                [32-bit]
long int             [64-bit]
long long int      [128-bit]
```

Varían de una plataforma a otra

Podemos saber el tamaño de los valores en < limits.h >

Rango para `int` está entre
`INT_MIN` y `INT_MIN` más `UINT_MAX` sin signo

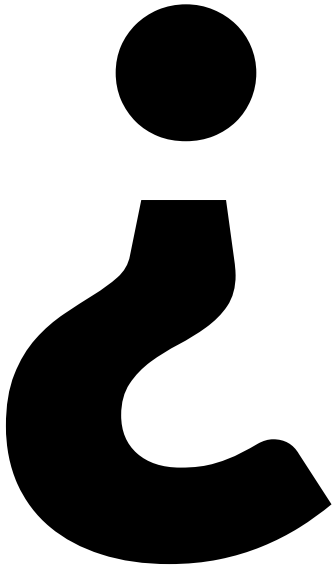
Rango para `long`, entre
`LONG_MIN` y `LONG_MIN` más `ULONG_MAX` sin signo

Hay constantes para cada tipo de número entero

Números enteros de tamaño específico en <stdint.h>

con signo	sin signo
intmax_t	uintmax_t
int8_t	uint8_t
int16_t	uint16_t
int32_t	uint32_t
int64_t	uint64_t

Para tipos de un tamaño específico



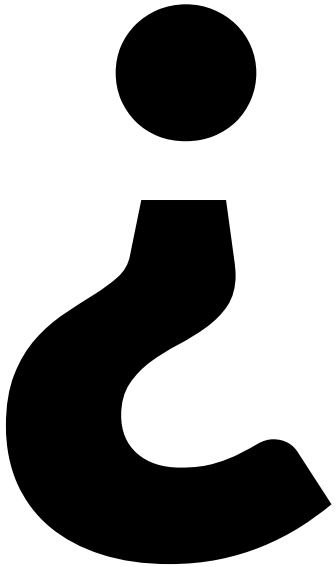
Como le damos un valor



Con los *literales*

```
0b1010
0173
1234
0xCAFE
```

- binario (0b)
- octal (0)
- decimal
- hexadecimal (0x)



hay decimales

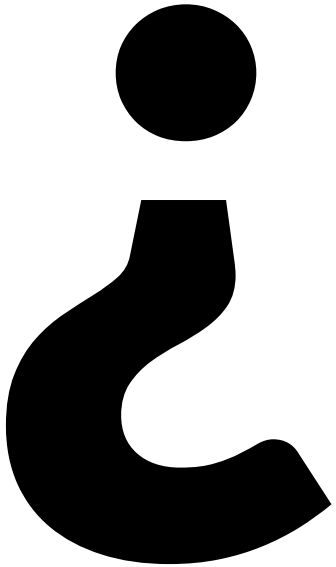


Números reales

```
float (4-bytes)
double (8-bytes)
long double
    (16-bytes)
```

```
1.0f
1.0
1.0F
```

{ La aritmética de punto flotante es mucho más compleja de lo que parece }



Algún otro tipo de dato



Caracteres individuales

tipo: `char` [8-bits]

literal: `'a'`

stdbool.h

Valores lógicos

tipo: `bool`

literal: `true` / `false`

> C99

¡A la terminal!



Sintácticamente correcto, gramaticalmente no.

```
int a = 4.0f;
```


¿Preguntas?

No hay strings 

UNRN Universidad Nacional de Río Negro

Son algo diferente

que vamos a ver en detalle más adelante

PERO

Antes de seguir, necesitamos *ver algo* *de cadenas*

Salida por consola

UNRN Universidad Nacional de Río Negro

printf: <stdio.h>

```
printf("cadena literal");
printf("cadena de formato", variables);
```

Forma básica de códigos de formato

«%?»

? puede ser

d ➳ int

c ➳ char

f ➳ double (y float)

c ➳ char

Sin los «» , por cada variable a mostrar en la consola

Hay un apunte de este tema

Un ejemplo

```
int valor = 10;
printf("numero en valor, %d\n", valor);
```

Se reemplaza en donde están los códigos de formato

`printf` admite varios valores!

```
int valor = 10;
double otro = 3.1456;
printf("v=%d, o=%f\n", valor, otro);
```

Se reemplazan en el mismo orden

¡A la terminal!



scanf

Tomando de la consola

De a una variable por vez

```
int variable;

scanf("%d", &variable);
```



Ya vamos a ver que es el &variable

Usa los mismos marcadores de tipo que `printf`

Los más importantes (lista no exhaustiva)

`%d` `%i`	`int`	Números enteros
`%u`	`unsigned int`	Números enteros sin signo

`%e`
`%f`
`%g`
	`float/double`	Números decimales

`%c`	`char`	Caracter individual

No exactamente igual a `printf`.


¿Preguntas?

Expresiones

UNRN Universidad Nacional de Río Negro

Asignación

UNRN Universidad Nacional de Río Negro

No se olviden



```
variable = expresion;
```

Formalmente, son las partes izquierda y derecha

L-VALUE = **R-VALUE** ;



L-VALUE

es el destino

Siempre y exclusivamente una variable

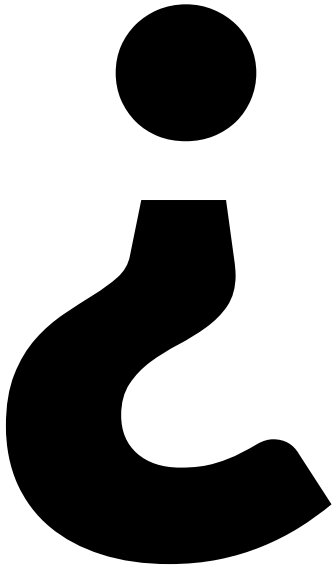
R-VALUE

es el “origen”

¡Tienen que ser del mismo tipo!

{ Por ahora }





¿Qué puede ser un R-VALUE?



R-VALUE

1. Variables
2. Literales
3. Operaciones
4. una mezcla de lo anterior

Operadores

UNRN Universidad Nacional de Río Negro

Aritméticos binarios

```
n + m
n - m
n / m
n * m
n % m
```

Aritméticos unarios

```
variable++;     //incremento
variable--;     //decremento
-variable;        //inversión de signo
```

¡A la terminal!



Un espacio antes y después de cada operador

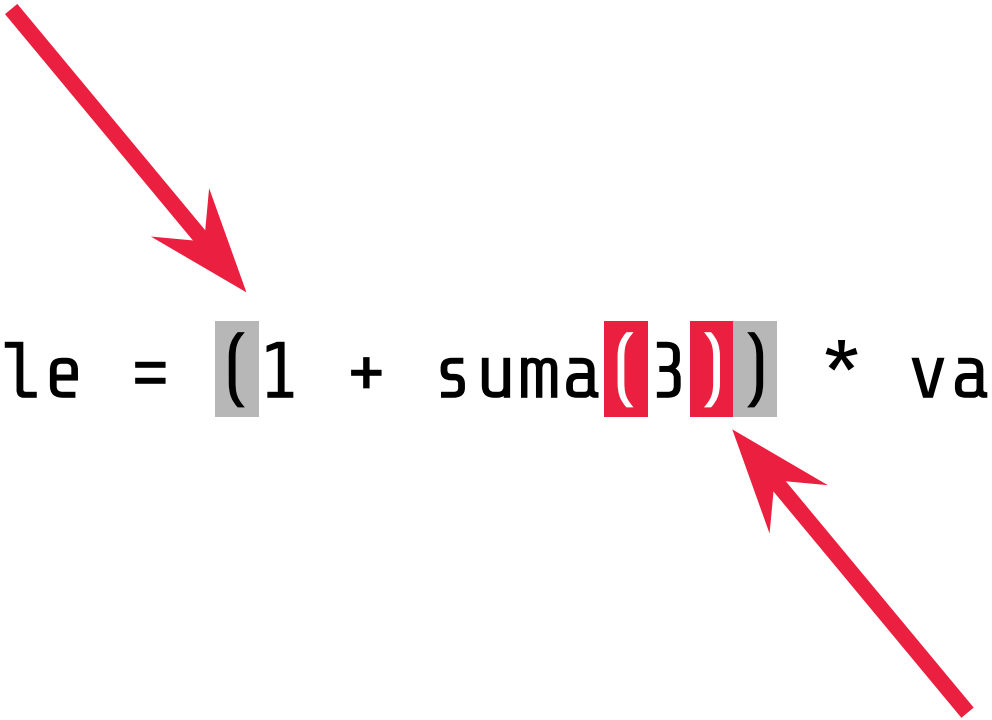

¿Preguntas?

Porque la idea es que se pueden combinar

```
variable = 1 + suma(3) * variable;
```

Y agrupar instrucciones con paréntesis

¡No hacen lo mismo! [cuidado con la precedencia]



```
variable = (1 + suma(3)) * variable;
```

Uno agrupa instrucciones y ***el otro los argumentos***

{ Cada operación individual, tiene una entrada y salida }

{ En donde ambos tienen un tipo }

¿Qué resultado obtenemos?

```
int a = 10;
float b = 2.33;
int c = a * b;
```

1. 23,3
2. 23,9999
3. 23
4. 0

Para empezar, mezclar tipos no *es una buena idea*

¿Qué resultado obtenemos?

```
int a = 10;
float b = 2.33;
double c = a * b;
```

1. 23,3
2. 23,9999
3. 23
4. 0

Mezclar tipos no *es una buena idea*


¿Preguntas?

Operadores relacionales

UNRN Universidad Nacional de Río Negro

```
n > m
n < m
n >= m
n <= m
n == m
n != m
```

} int



Dan como resultado un número entero `int`

{ Los booleanos son *técnicamente* nuevos }

¿Pero qué significa?

numero > 4



numero >= 4



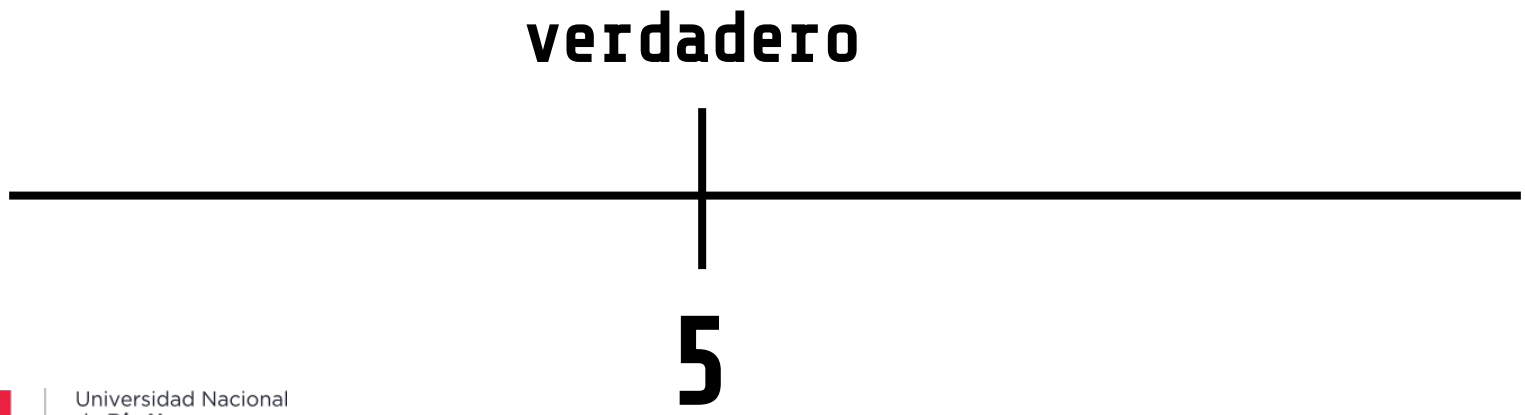
numero < 7



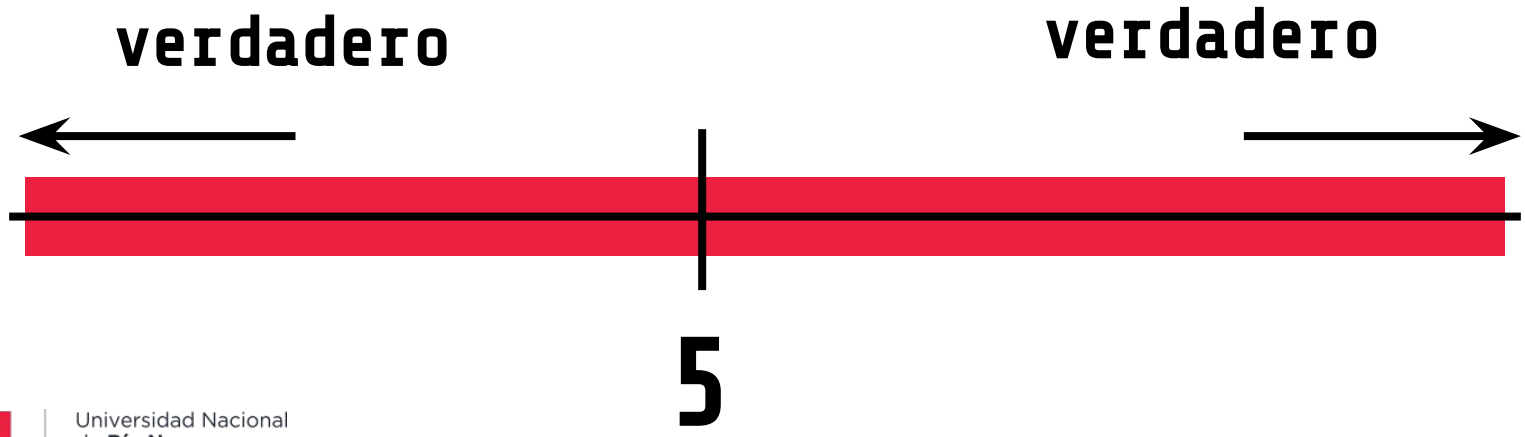
```
numero <= 7
```



```
numero == 5
```



```
numero != 5
```



0 es falso
1 es verdadero*

*Cualquier otra cosa que no sea cero es verdadero

Operadores lógicos

para condiciones más complejas

UNRN Universidad Nacional de Río Negro



negación
y lógico
o lógico

Un ejemplo rápido

al que volveremos en instantes nada más

```
(nota >= 4) && (nota<7)
```



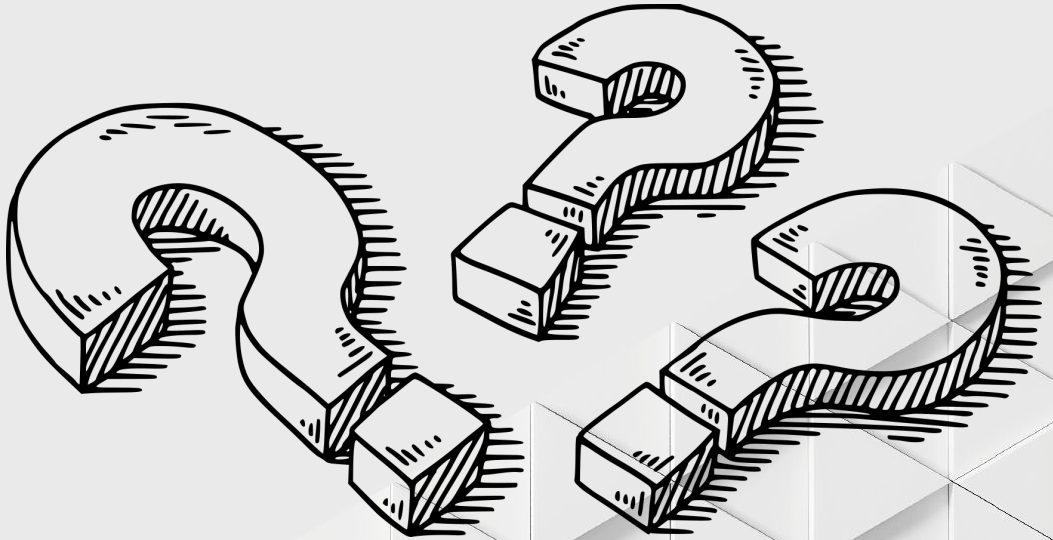

¿Preguntas?

Y esto...

¿Para qué?

Tomar decisiones

(condicionales)



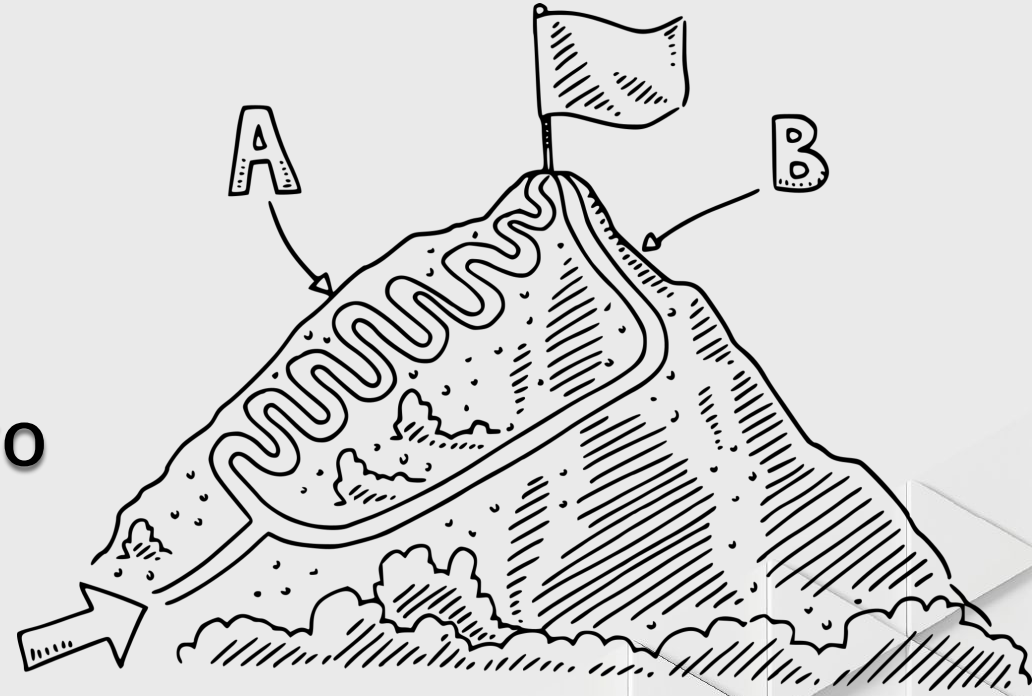
Estructuras de control

y el estilo a usar

UNRN Universidad Nacional de Río Negro

`if` simple

un camino alternativo



```
if (valor_booleano)
{
    bloque verdadero;
}
```

```
if (nota >= 4)
{
    printf("Aprobado\n");
}
```

```
(nota >= 4) && (nota<7)
```



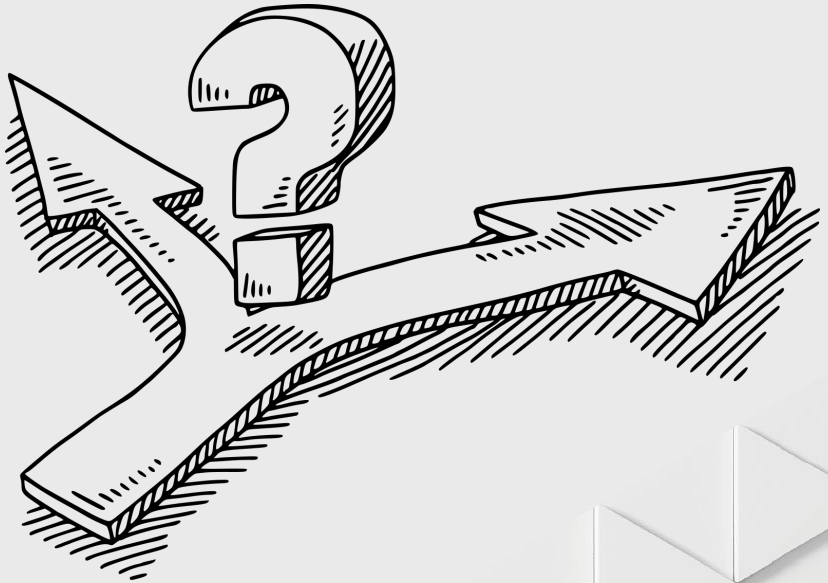
```
if ((nota >= 4) && (nota<<7)))
{
    printf("Aprobado\n");
}
```

El valor_booleano puede ser una variable

```
bool comparar = (nota >= 4) && (nota < 7);
if (compara)
{
    printf(“Aprobado\n”);
}
```

if/else

dos caminos excluyentes



```
if (condición)
{
    bloque verdadero;
}
else
{
    bloque falso;
}
```

Al `valor_booleano` , mejor llamémoslo...

if/else if/else

varios caminos



```
if (condición)
{
    bloque condición verdadero;
}
else if(otra_condición)
{
    bloque otra_condición verdadero;
}
```

Error gramatical



```
int x = 5;
if (x = 0) {
    printf("cero\n");
} else {
    printf("no cero\n");
}
```

Asignación en lugar de comparación

Cuestiones a tener en cuenta:

UNRN Universidad Nacional de Río Negro

Todos los bloques van con llaves

y estás en línea propia

Sin olvidarnos qué `if` acepta `int`

```c
#include <stdio.h>
#include <stdbool.h>

int main ()
{
    bool b = true;

    if ( b )
    {
       printf ( "Si!\n" );
    }
    else
    {
       printf ( "No!\n" );
    }
    return 0;
}
```

Los `bool` no forman parte del C original

Se ejecuta solo un camino

Se ejecuta solo el primero que coincida

¡Cuidado con las superposiciones!

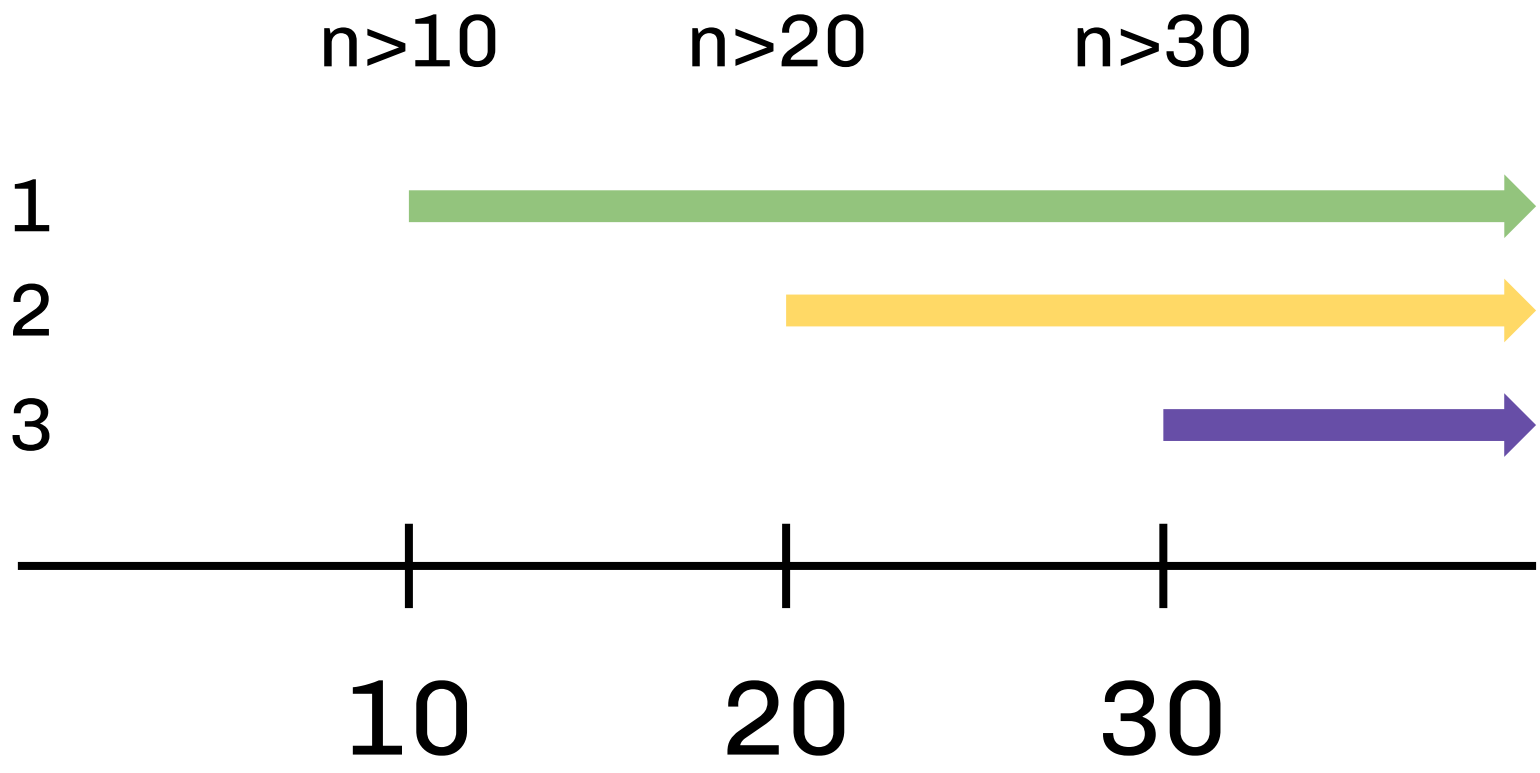
```
int numero = scanf(“%d”);

if (numero >= 0)
{
    printf(“Es cero”);
}
else if (numero < 4)
{
    printf(“menor a 4”);
}
else
{
     printf(“que va aca?”);
}
```

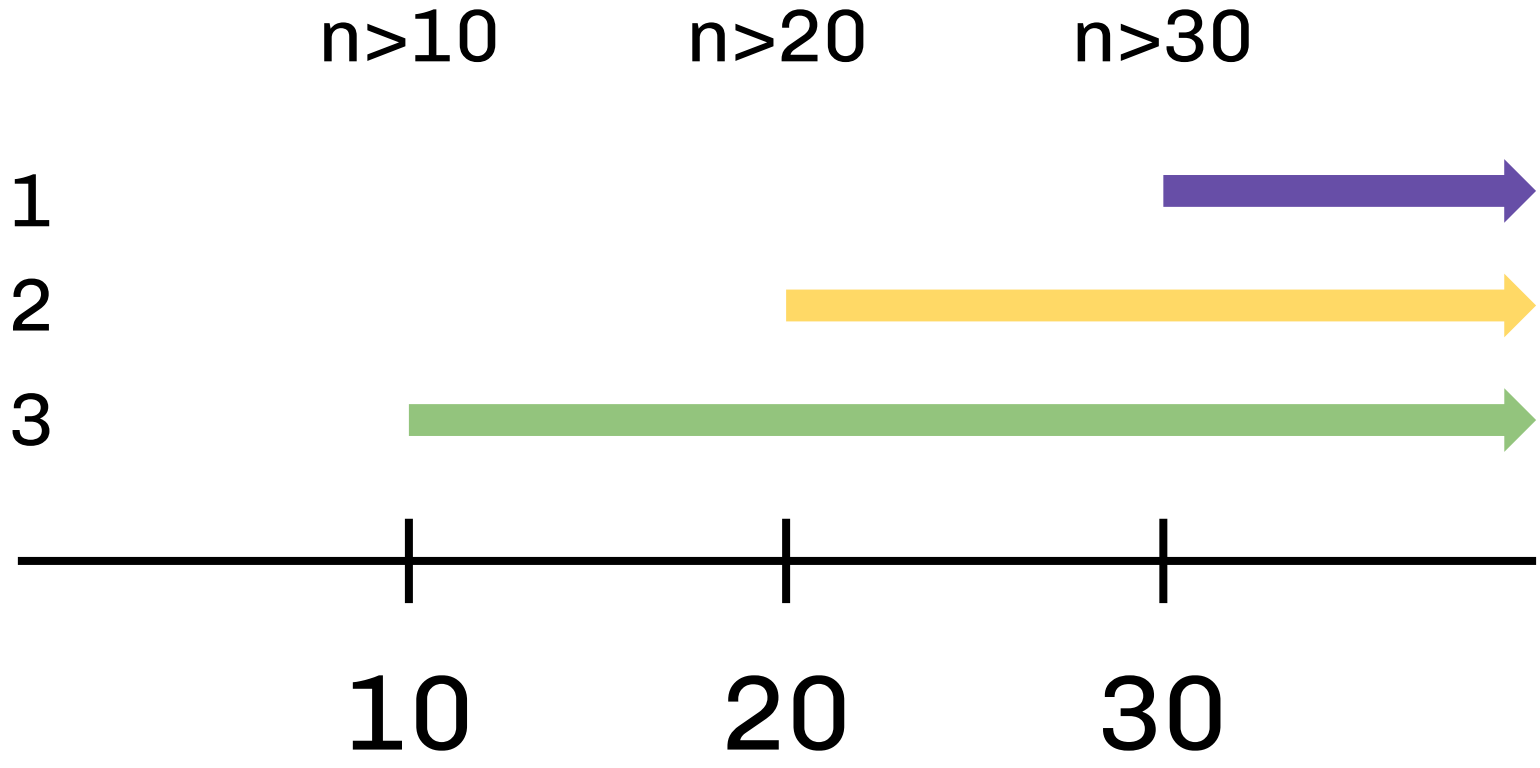
Pueden cambiar el significado del condicional

```
numero >= 0
```



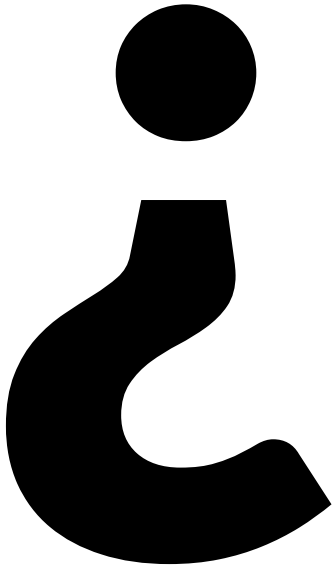

n>10
n>20
n>30
1
2
3
10
20
30

```
print("antes")
if n > 10:
    print("mayor que 10")
elif n > 20:
    print("mayor que 20")
elif n > 30:
    print("mayor que 30")
print("despues")
```


n>10
n>20
n>30
1
2
3
10
20
30

```
print("antes")
if n > 30:
    print("mayor que 30")
elif n > 20:
    print("mayor que 20")
elif n > 10:
    print("mayor que 10")
print("despues")
```

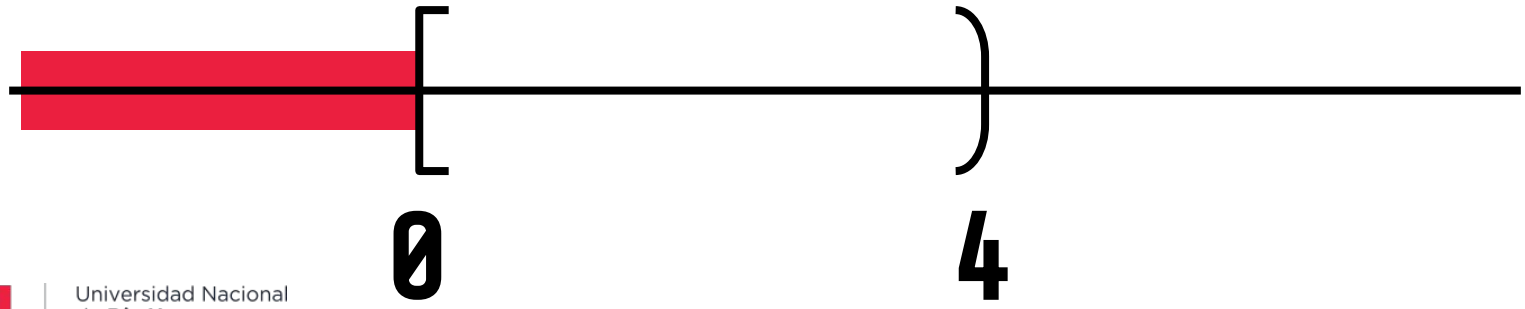

¿Preguntas?



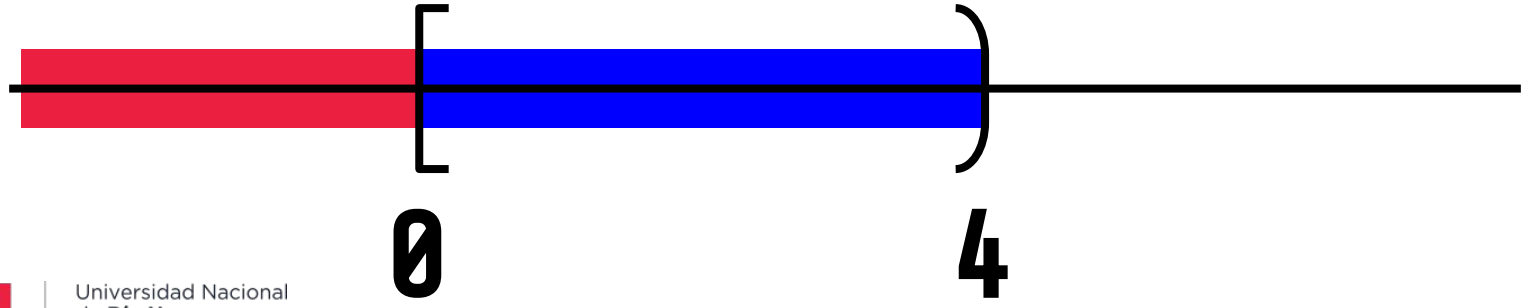
Como solucionarlo



```
!(numero >= 0)
```



numero < 4




¿Preguntas?

¿Cuantos caminos nos da una condición individual?

Con la condición compuesta y más compleja

```
if ((numero >= 4) && (numero < 7)){
```

Solo obtenemos un resultado


¿Preguntas?

¡A la terminal!



Ejemplo en C: número par

```c
#include <stdio.h>

int main(void) {
    int n = 0;
    int r = 0;
    printf("Ingrese un número: ");
    scanf("%d", &n);

    r = n % 2;
    if (r == 0) {
        printf("%d es par\n", n);
    } else {
        printf("%d es impar\n", n);
    }

    return 0;
}
```



A1

Trabajo en equipo #1

Resolución en Parejas (15 minutos)

La universidad requiere un pequeño módulo de validación para la inscripción a un taller avanzado. El sistema debe pedir al usuario:

1. Su **edad** (número entero).
2. Su **promedio académico** (número decimal).
3. La **cantidad de materias aprobadas** (número entero).

El programa debe calcular y mostrar por pantalla:

- Un mensaje de bienvenida indicando la edad recibida.
- Si el estudiante cumple los requisitos para inscribirse: **debe ser mayor o igual a 18 años Y tener al menos 10 materias aprobadas**.
- En caso de cumplir, mostrar: `"Inscripcion aceptada. Promedio registrado: [promedio]"`.
- En caso contrario, mostrar: `"No cumple con los requisitos mínimos."`.

Ayudín

```c
#include <stdio.h>

int main() {
    int entero;
    float decimal;

    // Input: scanf("%d", &entero);
    // Output: printf("Texto %f", decimal);

    return 0;
}
```

Revisión en Parejas (15 minutos)

1. **Estructura base:** ¿Incluye `#include <stdio.h>`, `int main()` y `return 0;`?
2. **Declaración de variables:** ¿Todas las variables tienen su tipo explícito (`int`, `float`) antes de usarse?
3. **Puntos y comas `;`:** ¿Casi todas las líneas de instrucción terminan en ; (excepto `#include`, `main` e `if`)?
4. **Uso de `scanf`:** ¿Utilizaron los especificadores correspondientes (`%d`, `%f`) y el símbolo `&` antes de la variable?
5. **Bloques de código:** ¿Las condiciones `if` / `else` encierran sus bloques entre llaves `{ }`?
6. **Lógica:** ¿La condición lógica (`&&`) evalúa correctamente la edad y las materias?

Tarea del Revisor: Si encuentran un error, **marquen la línea con un círculo** y escriban una nota al pie explicando qué fallaría al intentar compilar o ejecutar.


¿Preguntas?

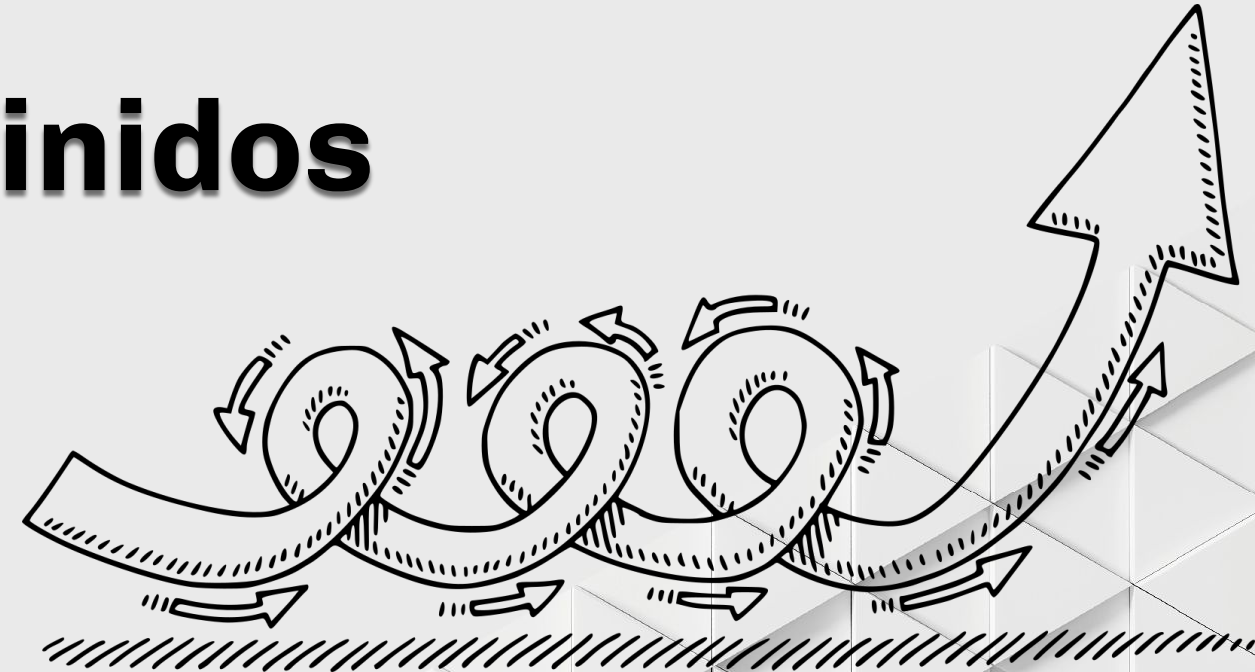
{ El jueves lo pasan a la computadora ;-) }

Repetiéndonos



UNRN Universidad Nacional de Río Negro

Lazos definidos



```
for (inicio; consulta; vuelta)
{
    bloque;
}
```

Lazo definido `for`

```
for (int i = 0; i < 10; i++)
{
    1️⃣
    bloque;
}
2️⃣
```

```
while (condición)
{
    bloque;
}
```

Lazo indefinido `while`

```
int i = 0;
while(i < 10🅰️)
{
    1️⃣
    i++;
}
2️⃣
```

for y while
son intercambiables*

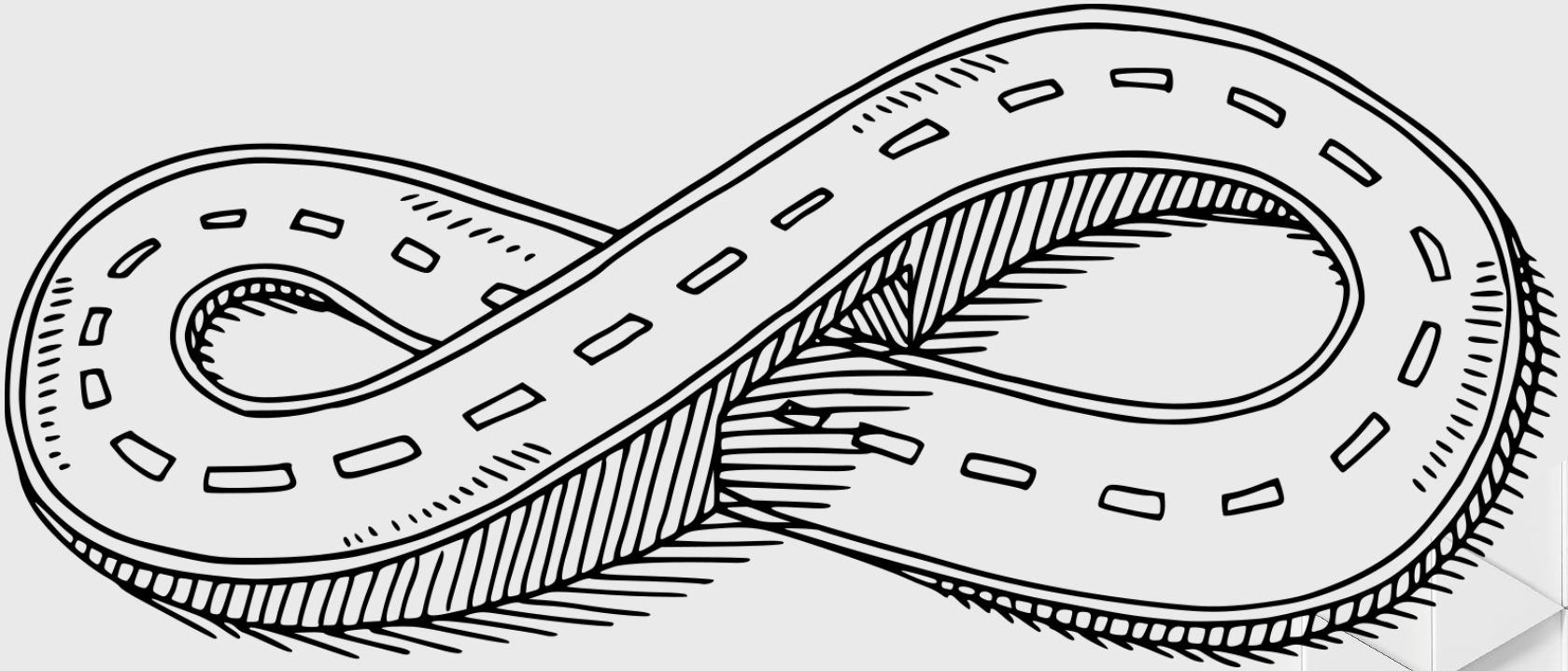
*por lo general

```
do
{
    bloque;
}
while (condición);
```

Lazo indefinido do..while

```
int i = 0;
do
{
    1️⃣
    i++;
}
while (i < 10🅰️);
2️⃣
```


while (not edge) {
run();
}
do {
run();
} while (not edge);



Lazos más comunes

UNRN Universidad Nacional de Río Negro

De menor a mayor

```
int numero = 0;
while (numero < 10)
{
    printf("numero:%d", numero);
    numero = numero + 1;
}
```

Este es uno de los más comunes.

De menor a mayor

```
int numero = 0;
do
{
    printf("numero:%d", numero);
    numero = numero + 1;
}
while (numero < 10);
```

Este es uno de los más comunes.

De mayor a menor

```
int numero = 10;
while (numero > 0)
{
    printf("numero:%d", numero);
    numero = numero - 1;
}
```

Cuando tenemos menos idea de cuantas veces ejecutar

```
int numero = 10;
bool bandera = true;
while (bandera)
{
    printf("numero:%d", numero);
    numero = numero - 1;
    if (numero > 0)
    {
        bandera = false;
    }
}
```

¡El `if` interno puede ser mucho más complejo!

¡La condición puede ser compuesta!

```
int numero = 10;
while (((numero > 0) && (numero < 20)))
{
    printf("numero:%d", numero);
    numero = numero + 1;
    if (numero % 2 == 0)
    {
        numero = -numero * 2;
    }
}
```

Aunque este ejemplo sea *un poco rebuscado* :-)

Suma de una cantidad abierta de números

```
int numero;
int cantidad = 0;
int suma = 0;
bool activo = true;
while (activo)
{
    printf(“numero %d”, cantidad);
    scanf(“%d”, &numero);
    cantidad = cantidad + 1;
    if ((cantidad > 100) && (numero < 0))
    {
        activo = false;
    }
}
```

¡El `if` interno puede ser mucho más complejo!

Manipulaciones de lazos

UNRN Universidad Nacional de Río Negro

break

```
int i = 0;
while(i < 10)
{
    break;
    i++;
}
printf("i es %d\n", i); 1️⃣
```

`break;`
para cortar el lazo

`continue ;`
para pasar a la siguiente iteración

Los lazos sin `break` y `continue` en su lugar, lazos con bandera

Prefieran usar `while` en lugar de `for`

Deciciones multiples

UNRN Universidad Nacional de Río Negro

```
switch (expresión)
{
    case valor:
        sentencia
    break;
    default:
        sentencia;
}
```



```
switch (expresión)
{
    case valor:
        sentencia
    break;
    default:
        sentencia;
}
```



¡son opcionales!

El no usar break agrupa casos

```
switch (numero)
{
     case 1:
     case 2:
     case 3:
     case 4:
     case 5:
     case 6:
          printf ("numero es >= 1 y <= 6\n");
          break;
}
```

El no usar break agrupa casos

```
int veces = 0;
switch (numero)
{
     case 1:
        veces = veces + 10;
     case 2:
        veces = veces + 10;
     case 3:
         veces = veces + 10;
         break;
}
printf ("veces es %d\n", veces);
```

¿Que valores acepta?
¿Cuál es la salida?

¡A la terminal!




¿Preguntas?

Resolución en Parejas (15 minutos)

El departamento de sistemas de un club deportivo necesita una pequeña aplicación para procesar los tiempos de entrenamiento de sus atletas.

Escriban un programa en C que realice lo siguiente:

1. **Validación de entrada:** Solicite al usuario la cantidad N de atletas a evaluar. El programa debe verificar que N sea un número positivo ($N>0$). Si el usuario ingresa un valor menor o igual a cero, debe volver a pedirlo en bucle hasta que ingrese un valor válido (usando `while` o `do-while`).
2. **Carga de datos y acumulación:** Utilizando un lazo `for`, el programa debe pedir consecutivamente el tiempo en segundos (número flotante) de cada uno de los N atletas.
3. **Cálculo de estadísticas:** Durante o al finalizar la lectura, el programa debe determinar:
    - El **tiempo promedio** del equipo.
    - La **cantidad de atletas** que superaron la marca de calificación (tiempo menor a 60.0 segundos).
4. **Resultado:** Mostrar por pantalla el promedio general y el total de atletas calificados.

Ayudín II

```
// Bucle while (valida mientras la condición sea verdadera)
while (condicion) { ... }

// Bucle do-while (ejecuta al menos una vez)
do {
    ...
} while (condicion);

// Bucle for (inicialización; condición; incremento)
for (int i = 0; i < limite; i++) {
    ...
}
```

Revisión en Parejas (15 minutos)

1. **Inicialización:** ¿Los acumuladores (ej. `suma = 0.0`) y contadores (ej. `calificados = 0`) se inicializan antes de entrar a los lazos?
2. **Validación ($N>0$):** ¿El bucle `while` o `do-while` repite la solicitud correctamente cuando $N<0$?
3. **Sintaxis del `for`:** ¿El bucle `for` tiene sus tres secciones separadas por punto y coma (`;`) y la variable de control se incrementa adecuadamente?
4. **Acumulación e incremento:** ¿Se utiliza correctamente `+=` o `suma = suma + valor` y `i++` dentro de los lazos?
5. **Condición de corte:** ¿El lazo no genera un bucle infinito por olvido de actualización de variables?
6. **Promedio fuera del lazo:** ¿El cálculo del promedio se realiza **después** de completar el bucle `for` (y no en cada iteración)?


¿Preguntas?

{ El jueves lo pasan a la computadora ;-) }

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar